Usuarios en Room y normalización de la base

Qué cambió

La base pasa de versión 2 a 3. Los usuarios se guardan en Room, el login consulta sus filas y las oportunidades dejan de guardar copias de organización, contacto y responsable. Las pantallas conservan el flujo conocido.

Este cambio completa la estructura relacional de las entidades que pide el checklist, bajo el modelo actual: cada oportunidad contiene los antecedentes de un evento y puede tener una organización y un contacto principal. No agrega cámara, calendario, animaciones ni dashboard de atrasos.

Antes y después

Antes	Ahora	Motivo
Login compara cuatro nombres escritos en Kotlin	UsuarioRepository consulta usuarios en Room	La identidad y el rol salen de datos persistidos
admin implica supervisor por su nombre	El rol viene de UsuarioEntity.rol	El nombre de usuario no determina sus permisos
responsable como texto en oportunidad	responsableId con clave foránea	No duplicar el login en cada registro
Nombre, tipo y rubro copiados en oportunidad	organizacionId	Una sola fila define los datos de organización
Datos de contacto copiados en oportunidad	Relación oportunidad_contacto	Consultar el contacto original por ID
Autor del seguimiento como texto	responsableId en seguimiento	Conservar identidad del autor aunque cambie su login
Contactos filtrados por nombre de organización	Filtrados por organizacionId	Dos organizaciones pueden llamarse igual
Panel usa lista fija de ejecutivos	Consulta ejecutivos activos de Room	La selección corresponde a usuarios reales de la base local

Tablas y relaciones

Origen	Relación	Destino
Usuario	Un usuario puede tener muchas oportunidades; cada una tiene un responsable.	Oportunidad
Usuario	Un usuario puede registrar muchas gestiones; cada gestión tiene un autor.	Seguimiento
Organización	Puede tener muchas oportunidades; una oportunidad puede no tener organización conocida.	Oportunidad
Organización	Una organización puede tener muchos contactos; cada contacto pertenece a una organización.	Contacto
Oportunidad	Puede tener cero o un enlace a su contacto principal.	OportunidadContacto
Contacto	Puede ser contacto principal de varias oportunidades.	OportunidadContacto
Oportunidad	Puede tener muchas gestiones; cada gestión pertenece a una oportunidad.	Seguimiento

UsuarioEntity

Guarda id, login único, nombre, hash de contraseña, rol y estado activo. Las cuentas iniciales siguen siendo admin, ejecutivo1, ejecutivo2 y ejecutivo3; todas tienen contraseña de prueba 1234.

Las cuentas iniciales se insertan si no existen. No se restablece el rol ni la contraseña cada vez que se abre el login. La contraseña escrita por el usuario se transforma a hash y se compara con la fila encontrada. Este mecanismo sencillo es para la demostración local, no una autenticación productiva.

La sesión contiene id, usuario y rol, pero no contraseña ni hash. Se conserva solo mientras vive el proceso; Room conserva los datos después de cerrarlo.

GestionOportunidadEntity

Guarda los datos propios de la oportunidad y su evento, responsableId y organizacionId. La organización puede ser null si todavía se desconoce. El responsable debe existir.

Ya no contiene nombreOrganizacion, rubroOrganizacion, correoContacto ni las otras copias. Cambiar un nombre en el catálogo se refleja en la siguiente consulta del detalle: no hay que actualizar todas las oportunidades.

OrganizacionEntity y ContactoEntity

Mantienen los datos de catálogo. El contacto ya tenía organizacionId; ahora se utiliza esa relación también al elegirlo desde Crear. El nombre es una etiqueta, no una clave de identidad.

Por qué hay OportunidadContactoEntity

El formulario admite un contacto principal opcional. Su asociación queda en una fila con oportunidadId como clave primaria y contactoId como clave foránea. No elegir contacto significa no tener esa fila.

La oportunidad puede registrar una organización antes de conocer al contacto. Al separar la selección del contacto se evita duplicar dentro de la fila de oportunidad la dependencia contacto → organización.

Repository comprueba además que el contacto pertenezca a la organización elegida.

La clave primaria oportunidadId limita a un contacto principal por oportunidad. Para admitir varios habría que cambiar esa clave por una compuesta y adaptar el formulario; no se implementó ese alcance adicional.

SeguimientoEntity

Guarda oportunidadId y responsableId junto a acción, resultado, fecha y observaciones. Reasignar la oportunidad no cambia el autor de gestiones anteriores. Las acciones y resultados son hechos históricos, no campos utilizados para relacionar tablas.

Cómo se muestran los nombres sin copiarlos

OportunidadDao usa JOIN entre las tablas. OportunidadDetalle recibe el resultado y se entrega a la pantalla. Es un objeto en memoria, no una tabla nueva.

```
Fila de oportunidad: responsableId = 2, organizacionId = 7
                ↓ JOIN
Fila de usuario 2: ejecutivo1
Fila de organización 7: Organización de ejemplo
                ↓
OportunidadDetalle: datos propios + etiquetas actuales
```

Embedded agrupa las columnas propias de la oportunidad. Los getters permiten seguir usando detalle.nombreOportunidad sin duplicar ese dato en disco. Se marcan Ignore para que Room no intente tratarlos como columnas adicionales.

SeguimientoDetalle hace lo mismo con la gestión y el login de su autor. La tabla de seguimiento no almacena ese nombre.

Recorrido de un login

1. LoginViewModel valida que ambos campos tengan contenido.
2. La corrutina llama UsuarioRepository.autenticar.
3. Se preparan las cuentas iniciales que falten y se busca por login.
4. Se comprueban cuenta activa, rol conocido y contraseña.
5. SesionActual.iniciar conserva identidad y rol de la fila.
6. El callback abre Inicio. Si falla se mantiene el formulario y se muestra el error.

El nombre admin ya no es una condición de permisos. Una cuenta con otro nombre puede tener rol supervisor en la tabla y se reconocerá como tal.

Recorrido de un guardado

1. CrearUiState conserva campos y los IDs seleccionados.
2. CrearViewModel valida y construye la entidad sin los textos de catálogo.
3. Repository lee el usuario activo en Room y comprueba acceso.
4. Si hay contacto, comprueba su existencia y pertenencia a la organización.
5. Una transacción inserta/actualiza oportunidad y su enlace al contacto.

6. El detalle y listado consultan los JOIN y muestran las etiquetas.

Las lecturas de catálogos de CrearViewModel también pasan ahora por CatalogoRepository. No necesita llamar directamente al DAO.

Permisos después del cambio

Repository no confía únicamente en el rol recordado por la pantalla. Consulta el usuario por ID al escribir y obtener detalles. Si se desactiva una cuenta, no puede seguir guardando aunque una pantalla conserve botones antiguos.

El listado combina la consulta de oportunidades y la fila de usuario observada. Cuando cambia el usuario, vuelve a calcular su alcance. El historial también cruza sus emisiones con el acceso actual para no entregar nuevas gestiones después de perder acceso por reasignación.

El panel lista ejecutivos activos y pasa un ID a asignar. Repository comprueba nuevamente que el destinatario sea un ejecutivo activo. Cambiar estado utiliza la regla de transiciones con el rol leído de Room.

Migración de datos anteriores

No se usa fallbackToDestructiveMigration. Los caminos registrados son 1 → 2 → 3 y 2 → 3. MigracionInicial conserva el SQL de la actualización anterior.

MigracionNormalizacion realiza estos pasos dentro de la migración transaccional de Room:

1. Lee las oportunidades y gestiones antiguas antes de reconstruir sus tablas.
2. Crea usuarios, oportunidades normalizadas, enlaces e historial relacionado.
3. Inserta las cuatro cuentas conocidas.
4. Busca la organización/contacto cuya información coincide exactamente con la copia antigua; si no existe, crea el registro que conserva esa información.
5. Sustituye los datos copiados por IDs y conserva los IDs originales de oportunidades.
6. Convierte autores del historial a usuarios relacionados, conservando IDs de gestiones.
7. Comprueba PRAGMA foreign_key_check para detectar referencias inválidas.

Se compara el conjunto de datos, no solamente el nombre. Dos organizaciones con igual nombre y distinto rubro no se mezclan. Si un autor antiguo no es una cuenta conocida, se crea inactivo y sin contraseña utilizable. Así conserva la autoría sin dar acceso nuevo.

Si una copia antigua tenía un contacto pero organización vacía, se conserva una organización con datos vacíos. No se inventa el nombre. Es un registro heredado que conviene completar/revisar después. Las diferencias entre copias antiguas pueden generar registros distintos; es intencional para no perder información ni adivinar identidades.

La migración trabaja con las filas en memoria porque es un MVP académico pequeño. Para volúmenes grandes convendría migración por lotes o más transformación SQL.

Orden recomendado de lectura

1. UsuarioEntity, UsuarioDao y UsuarioRepository.
2. LoginViewModel y SesionActual.

3. GestionOportunidadEntity, OportunidadContactoEntity y SeguimientoEntity.
4. OportunidadDao y OportunidadDetalle.
5. OportunidadRepository.
6. CrearViewModel: cargar, elegirOrganizacion, elegirContacto y crearRegistro.
7. AppDatabase y MigracionNormalizacion.
8. NormalizacionRoomTest.

Pruebas y límite de la verificación

El procesador de Room y la compilación Kotlin principal pasaron en la primera comprobación de esta normalización. El empaquetado del APK falló por archivos temporales bloqueados. No se alteraron atributos ni configuración para evitarlo.

La ejecución siguiente de pruebas no fue autorizada. Por eso NO se afirma que las nuevas pruebas instrumentales hayan pasado ni que se haya ejecutado la migración en el emulador. Después se ordenó CrearViewModel; ese último ajuste requiere volver a compilar en Android Studio.

Sí se compararon mediante SQLite las columnas, claves e índices del SQL de migración con el esquema que Room generó: coinciden las seis tablas. También se comprobó que SQLite rechaza referencias inexistentes. Esto valida la estructura, pero no sustituye probar la migración completa en Android.

NormalizacionRoomTest usa bases de nombre aleatorio y no toca agencia_atenas.db. Cubre persistencia de usuarios, rol leído de la tabla, cuentas inactivas, homónimos, lectura por JOIN, asignación y migración de una base v2 con historial.

Recorrido manual de aceptación

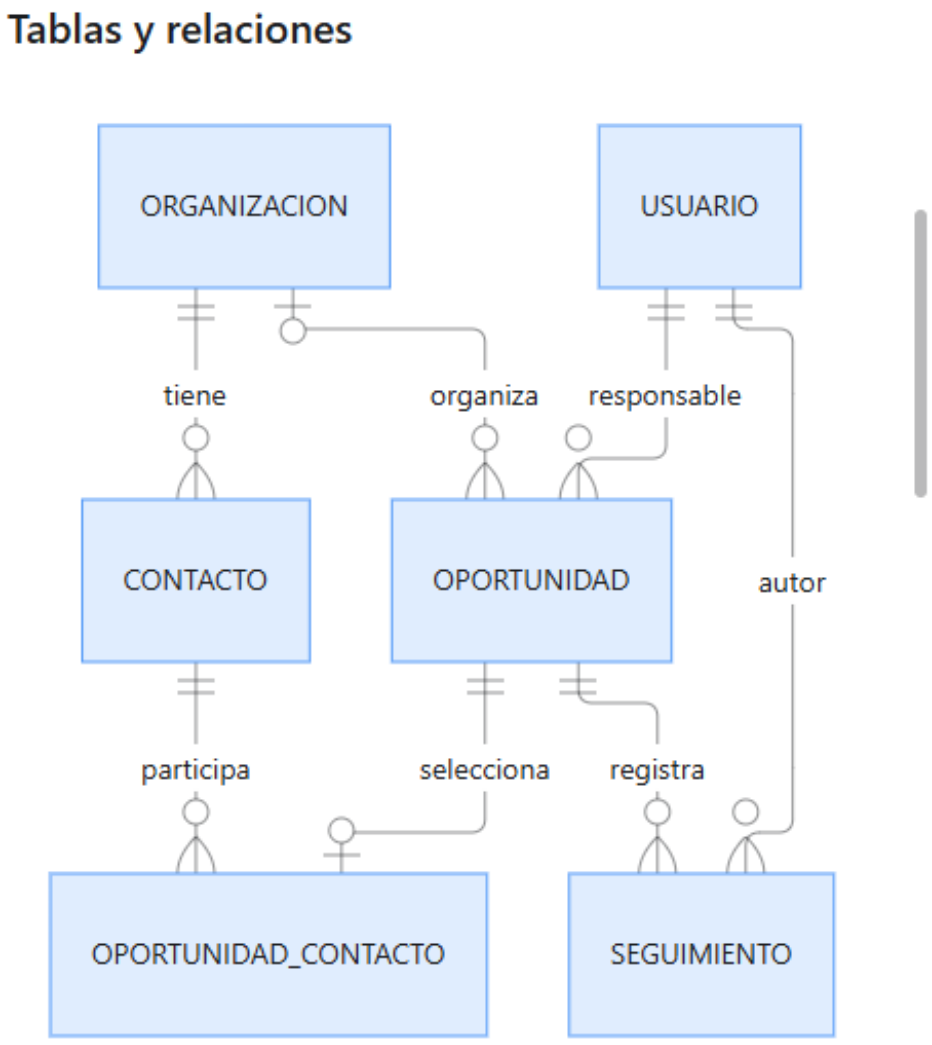
- Actualizar una instalación con datos de prueba anteriores, sin desinstalar.
- Ingresar con las mismas cuentas y comprobar datos/historial conservados.
- Crear dos organizaciones con el mismo nombre y distinto rubro, cada una con un contacto.
- Crear oportunidad eligiendo la segunda y comprobar que solo ofrece su contacto.
- Guardar/editar y revisar nombres, responsables y seguimiento.
- Reasignar y comprobar alcance de ejecutivo1/ejecutivo2.
- Cerrar el proceso, volver a ingresar y comprobar persistencia.
- Ejecutar NormalizacionRoomTest en un emulador de prueba antes de dar por cerrada la migración.

Qué sigue pendiente del checklist

Esta etapa no implementa cámara, calendario, animaciones, nuevas consultas de atrasos ni los demás filtros. Los usuarios ahora sí son entidades persistidas y se retiraron las copias de catálogos de la oportunidad. La defensa de 3FN debe explicar las dependencias de estas tablas y el supuesto de un evento por oportunidad, no limitarse a decir que existen claves foráneas.

Diagrama de relaciones

El diagrama complementa la tabla explicativa de relaciones incluida anteriormente.



Cómo leer los símbolos

Círculo: relación opcional (cero). Barra: uno. Pata de gallo: varios. Las combinaciones indican cero o uno, uno o varios, o cero o varios.

Ejemplo: una organización puede tener varios contactos; cada contacto pertenece a una organización. Una oportunidad puede no tener todavía un contacto principal.